МИНОБРНАУКИ РОССИИ

Санкт-Петербургский государственный

электротехнический университет

«ЛЭТИ» им. В.И. Ульянова (Ленина)

Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Построение и анализ алгоритмов»

Тема: **Поиск подстроки в строке.**

Студент гр. 4343 **Лапшин А.Е.**

Преподаватель **Жангиров Т.Р.**

Санкт-Петербург

2026 г.

ЗАДАНИЯ

Задание 1

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T.

Вход:

Первая строка – P

Вторая строка – T

Выход:

индексы начал вхождений P в T, разделённые запятой; если P не входит в T, то вывести -1.

Входные данные

ab

abab

Выходные данные

0,2

Задание 2

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

Первая строка – P

Вторая строка – T

Выход:

Если A является циклическим сдвигом B, то индекс начала строки B в A; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Входные данные

defabc

abcdef

Выходные данные

3

ВЫПОЛНЕНИЕ РАБОТ

Алгоритм Кнута-Морриса-Пратта (КМП)

Алгоритм КМП предназначен для эффективного поиска всех вхождений строки-шаблона **P** в строку-текст **T**. Его основная идея заключается в использовании префикс-функции, которая позволяет избежать повторной проверки уже совпавших символов при возникновении несовпадения.

Временная сложность алгоритма составляет $\boldsymbol{O}(|\boldsymbol{P}| + |\boldsymbol{T}|)$

Пространственная сложность алгоритма составляет $\boldsymbol{O}(|\boldsymbol{P}| + |\boldsymbol{T}|)$

Префикс-функция

Префикс-функция **pi(i)** для строки **s** определяется как длина наибольшего собственного префикса строки **s[0…i]**, который одновременно является её суффиксом.

Алгоритм вычисления префикс-функции:

- Инициализируем массив pi нулями
- Для каждого i от 1 до $n - 1$:
  - Берём значение $\boldsymbol{j} = \boldsymbol{pi}[\boldsymbol{i} - \mathbf{1}]$ (длина предыдущего совпадения)
  - Пока $\boldsymbol{j} > \mathbf{0}$ и $\boldsymbol{s}[\boldsymbol{i}] != \boldsymbol{s}[\boldsymbol{j}]$, уменьшаем $\boldsymbol{j}$ до $\boldsymbol{pi}[\boldsymbol{j} - \mathbf{1}]$
  - Если $\boldsymbol{s}[\boldsymbol{i}] == \boldsymbol{s}[\boldsymbol{j}]$, увеличиваем $\boldsymbol{j}$ на 1
  - Записываем $\boldsymbol{pi}[\boldsymbol{i}] = \boldsymbol{j}$

Поиск подстроки

После вычисления префикс-функции для шаблона **P** выполняется проход по тексту **T**:

- Поддерживается переменная j — текущая длина совпадения
- При совпадении символов j увеличивается
- При несовпадении j уменьшается с использованием префикс-функции
- Когда j достигает длины шаблона, фиксируется вхождение

Задание 1

Используется описанная выше префикс-функция для построения массива pi для шаблона **P**. Затем выполняется поиск всех вхождений **P** в **T**. Индексы всех найденных вхождений собираются в вектор и выводятся на экран через запятую. Если вхождений нет, выводится -1.

Задание 2

Для определения, является ли строка A циклическим сдвигом строки B, используется тот же алгоритм КМП. Проверяется цикличнсоть условием, что Строка A является циклическим сдвигом B тогда и только тогда, когда A встречается как подстрока в $\boldsymbol{B} + \boldsymbol{B}$. Причем не создаётся явная удвоенная строка, а используется доступ по модулю длины строки (сам цикл идет до удвоенного значения длины строки $\boldsymbol{B}$. Возвращается индекс первого найденного вхождения.

ТЕСТИРОВАНИЕ

Задание 1

Таблица 1 - Тестирование задания 1

тест	Входные данные	Выходные данные
1	ab abab	0,2
2	aba abacaba	0,4
3	aa aaaaa	0,1,2,3
4	abc defdef	-1
5	a banana	1,3,5
6	(пусто) abc	-1
7	abab abababab	0,2,4
8	(пусто) (пусто)	0
	aba (пусто)	-1
	аасс асаааасссасаасс	3,10

Задание 2

Таблица 2 - Тестирование задания 2

тест	Входные данные	Выходные данные
1	defabc abcdef	3
2	abcabc abcabc	0
3	cab abc	1
4	aaaa aaaa	0
8	(пусто) (пусто)	0

	aba (пусто)	-1
	abffdcafd abffdcafdfdcafddcafd	-1

ПРИЛОЖЕНИЕ А

Код программы 1:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;


vector<size_t> prefix_function(string s) {
    size_t n = s.length();
    vector<size_t> pi(n);

    for (size_t i = 1; i < n; i++) {
        size_t j = pi[i-1];

        while ((j > 0) && (s[i] != s[j]))
            j = pi[j-1];

        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

vector<size_t> CMP(string s, string t, vector<size_t> pi) {
    size_t n_s = s.length();
    size_t n_t = t.length();
    vector<size_t> result;

    size_t j = 0;
    size_t i = 0;
    while (i < n_s) {

        if(s[i] == t[j]) {
            i++;
            j++;

            if(j == n_t) {
                result.push_back(i - n_t);
                j = pi[j-1];
            }
        }
        else {
            if(j != 0) {
                j = pi[j-1];
            }
            else {
                i++;
            }
        }

    }

    return result;
}
```

```
void print_answer(vector<size_t> result) {
    size_t n = result.size();

    if (n == 0) {
        cout << "-1" << endl;
        return;
    }

    for (size_t i = 0; i < n; i++) {
        cout << result[i];
        if (i != result.size() - 1) cout << ",";
    }
    cout << endl;
}

int main() {
    string t;
    string p;
    getline(cin, p);
    getline(cin, t);

    if ((t.empty() || p.empty()) && !(t.empty() && p.empty())) {
        cout << -1 << endl;
        return 0;
    } else if(t.empty() && p.empty()) {
        cout << 0 << endl;
        return 0;
    }

    vector<size_t> pref = prefix_function(p);

    vector<size_t> result = CMP(t, p, pref);

    print_answer(result);

    return 0;
}
```

Код программы 2:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

vector<size_t> prefix_function(string& s) {
    size_t n = s.length();
    vector<size_t> pi(n);

    for (size_t i = 1; i < n; i++) {
        size_t j = pi[i-1];

        while ((j > 0) && (s[i] != s[j]))
            j = pi[j-1];

        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

int CMP(string& s, string& t, vector<size_t>& pi) {
    size_t n_s = s.length();
    size_t n_t = t.length();

    size_t j = 0;
    size_t i = 0;
    while (i < n_s * 2) {

        if(s[i % n_s] == t[j]) {
            i++;
            j++;

            if(j == n_t) {
                return (i - n_t);
            }
        }
        else {
            if(j != 0) {
                j = pi[j-1];
            }
            else {
                i++;
            }
        }

    }

    return -1;
}

int main() {
    string t;
    string p;
    getline(cin, p);
```

```
    getline(cin, t);

    if (t.length() != p.length()) {
        cout << -1 << endl;
        return 0;
    }

    if (t.empty()) {
        cout << 0 << endl;
        return 0;
    }

    vector<size_t> pref = prefix_function(t);

    cout << CMP(p, t, pref) << endl;

    return 0;
}
```